Announcements

- Quiz 01 grades are published
- Start working on Project Milestone 3: Requirements
- Demo submission, commit each TODO

Web Engineering & Development (SWE 363)

Interactive Front-End Development

In today's lecture:

- Using JavaScript with HTML
- Document Object Model (DOM)
- Using third-party web APIs (JavaScript)
- JavaScript Object Notation (JSON)

Reference:

- Zybook: 5.1 to 5.4, 5.17

5.1 Using JavaScript with HTML

Using JavaScript with HTML

Key Concepts:

- **HTML** = structure, **CSS** = style, **JavaScript** = behavior
- Three ways to use JavaScript:
  - **inline** (`onclick="..."`)
  - **internal** (`<script>`)
  - **external** `.js` file (best practice)

```
<script src="script.js"></script>
```

Using JavaScript with HTML

The browser parses HTML **top to bottom**

```
<head>
    <script src="script.js"></script>          <!-- runs now: <button> doesn't exist yet -->
    <script src="script.js" defer></script>    <!-- runs after the HTML is parsed -->
</head>
<body>
    <button id="myButton">Click Me</button>
</body>
```

```
document.getElementById("myButton");  // without defer -> null -> TypeError
```

- `defer`: download in **parallel**, run **after** the DOM is built, in **order**
- Alternative:
  `document.addEventListener("DOMContentLoaded", () => { ... });`

5.2 Document Object Model (DOM)

What is the DOM?

The **DOM** is a **tree** of the page: every HTML element is a **node** JavaScript can **find, read, change, add, or remove**

```
<body>
  <h1>Welcome</h1>
  <p id="message">Hello World</p>
  <button>Click Me</button>
</body>
```

```
body
├── h1 ("Welcome")
├── p#message ("Hello World")
└── button ("Click Me")
```

Finding Elements

```
// By ID (single element)
const button = document.getElementById("myButton");

// First match of a CSS selector
const first = document.querySelector(".highlight");

// All matches of a CSS selector (NodeList)
const all = document.querySelectorAll(".highlight");

// Older alternatives (live HTMLCollection)
document.getElementsByClassName("item");
document.getElementsByTagName("p");
```

Check it exists before using it: `if (element) { ... }`

Changing Elements

```
const msg = document.getElementById("message");

// Content
msg.textContent = "Plain text";           // safe, preferred
msg.innerHTML = "<strong>Bold</strong>";  // parses HTML, use only when needed

// Style
msg.style.color = "red";
msg.style.backgroundColor = "blue";       // CSS kebab-case -> camelCase

// Attributes / properties
const link = document.getElementById("myLink");
link.href = "https://www.kfupm.edu.sa";
```

Adding and Removing Elements

```
// Create
const p = document.createElement("p");
p.textContent = "This is a new paragraph";

// Add to page
document.getElementById("container").appendChild(p);

// Remove
document.getElementById("toRemove").remove();
```

Examples:

50projects50days

5.3 Using Third-Party Web APIs

What is a Web API?

A **Web API** lets your page **request data from another service** over HTTP (weather, maps, news, ...)

Your page → request → **API server** → JSON response → **update the DOM**

Building the request URL

```
const url = `https://api.openweathermap.org/data/2.5/weather?q=${city}&appid=${apiKey}&units=metric`;

// city name to coordinates:
// https://nominatim.openstreetmap.org/search?q=${cityName}&format=json&limit=1
// weather data:
// https://api.open-meteo.com/v1/forecast?latitude=${lat}&longitude=${lon}&current_weather=true
```

`fetch()` and Promises

`fetch()` is **asynchronous**: it returns a **Promise** that resolves later with the response

```
fetch("https://api.example.com/data")
  .then(response => response.json())   // parse body as JSON (also a Promise)
  .then(data => console.log(data))     // use the data
  .catch(error => console.log("Request failed", error));
```

async / await

Same thing, easier to read:

```
async function getData() {
  try {
    const response = await fetch("https://api.example.com/data");
    if (!response.ok) { throw new Error(`HTTP ${response.status}`); }

    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.log("Something went wrong:", error.message);
  }
}
```

Status codes:

200 OK **401** Unauthorized (bad API key) **404** Not Found **500** Server Error

5.4 JavaScript Object Notation (JSON)

What is JSON?

A **text format** for exchanging data, language-independent, used by most web APIs

```
{ "name": "Ahmed", "age": 25, "isStudent": true, "courses": ["Math", "Physics"] }
```

Rules (vs. JS objects):

- Property names in **double quotes**; strings in **double quotes** (no single quotes)
- **No trailing commas**, no comments, no functions
- Allowed types: string, number, boolean, `null`, array, object

Converting JSON

```
// JSON string -> JavaScript object
const person = JSON.parse('{"name": "Ahmed", "age": 25}');
console.log(person.name);  // "Ahmed"

// JavaScript object -> JSON string
const json = JSON.stringify({ name: "Ahmed", age: 25 });
console.log(json);         // '{"name":"Ahmed","age":25}'
```

`JSON.parse` **throws** on invalid JSON, so wrap it in `try/catch`

Safe Property Access

The Problem

```
// What if the API doesn't return expected data?
const temperature = weatherData.main.temp; // Error if main is undefined!
```

The Solution: Optional Chaining

```
// Safe access - won't crash if property doesn't exist
const temperature = weatherData?.main?.temp;
const humidity = weatherData?.main?.humidity;

// With default values
const temperature = weatherData?.main?.temp ?? "Unknown";
const humidity = weatherData?.main?.humidity ?? 0;
```

30m

Demo

5.1 DOM Manipulation and API

API Key: 9c29da573838fd8cdd561179419142d7

API Key: d51f2f00c3b137ccfd135bd8f9dd50aa

Next Class

- React